

Problem 1

Simulate the following tasks and give your inferences and all the associated codes.

1. For $N = 5$, generate and plot a random undirected graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$ using Erdos-Renyi model.
 - (a) Report the adjacency matrix, degree matrix, incidence matrix, and Laplacian matrix.
 - (b) Compute and report the eigenvalues of Laplacian matrix. Report the algebraic connectivity λ_2 value. Explain the properties of the graph using the computed λ_2 quantity and mention why it is so important.
 - (c) Argue why high connectivity (i.e., each nodes having more connections) is a double-edged sword for agents in a network.
2. Assume the number of agents $N = 10^3$ and generate a random undirected connected graph (having spanning tree) $\mathcal{G}(\mathcal{V}, \mathcal{E})$ with N agents using Erdos-Renyi model and plot it.
 - (a) Highlight the node(s) with the highest number of neighbours with bright colour. Compute the mean degree, mean edges from the generated random graph and compare that with the theoretical value to see if it matches. Plot the diameter of the graph by colouring the corresponding edges.
 - (b) Pick any two nodes in random from the generated graph to compute the shortest path between them using any Shortest Path (SP) algorithm like Dijkstra algorithm and plot the shortest path between them in the graph.

Note: Both Matlab & Python have readily available package for plug and use, so that you need not write the code for the SP algorithm.

Solution:

1. Erdős-Rényi graph with $N = 5$

For $N = 5$ and edge probability $p = 0.5$, the generated Erdős-Rényigraph is shown below.

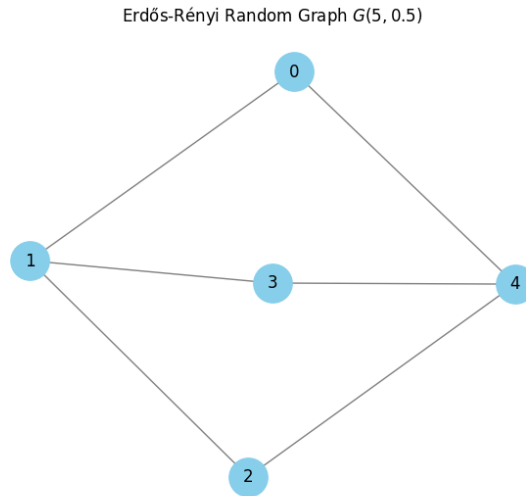


Figure 1: Generated Erdős-Rényi random graph $G(5, 0.5)$.

(a) Adjacency, Degree, Incidence, and Laplacian Matrices

The adjacency matrix of the generated graph is

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \end{bmatrix}$$

The degree matrix is obtained by placing the degree of each node along the diagonal:

$$D = \begin{bmatrix} 2 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 0 & 3 \end{bmatrix}$$

Using an arbitrary orientation for each edge, the incidence matrix is

$$B = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 \\ -1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 & 0 & 1 \\ 0 & -1 & 0 & 0 & -1 & -1 \end{bmatrix}$$

Finally, the Laplacian matrix is given by $L = D - A$, which gives

$$L = \begin{bmatrix} 2 & -1 & 0 & 0 & -1 \\ -1 & 3 & -1 & -1 & 0 \\ 0 & -1 & 2 & 0 & -1 \\ 0 & -1 & 0 & 2 & -1 \\ -1 & 0 & -1 & -1 & 3 \end{bmatrix}$$

(b) Laplacian Eigenvalues and Algebraic Connectivity

The computed eigenvalues of the Laplacian matrix are $\lambda(L) = \{0, 2, 2, 3, 5\}$. Hence, the algebraic connectivity is $\lambda_2 = 2$.

Since $\lambda_2 > 0$, the graph is connected. A larger λ_2 indicates stronger connectivity, while $\lambda_2 = 0$ indicates a disconnected graph.

Thus, λ_2 provides a measure of network connectivity and is important for analyzing information propagation and consensus in the network.

(c) Effect of High Connectivity

High connectivity is a double-edged sword for agents in a network. More connections allow information to propagate faster, improving coordination, cooperation, and consensus among agents. However, the same connections can also spread incorrect information, disturbances, or failures more rapidly throughout the network.

Thus, while higher connectivity improves information sharing and network robustness, it can also increase the propagation of undesirable effects.

2. Erdős-Rényi graph with $N = 10^3$

(a) Degree, Mean Degree, Mean Edges, and Diameter

For $N = 1000$ and $p = 0.01$, a connected Erdős-Rényi graph was generated. The nodes with the highest number of neighbours have degree

$$d_{\max} = 20,$$

attained by nodes 457 and 974. The corresponding nodes are highlighted in the generated graph.

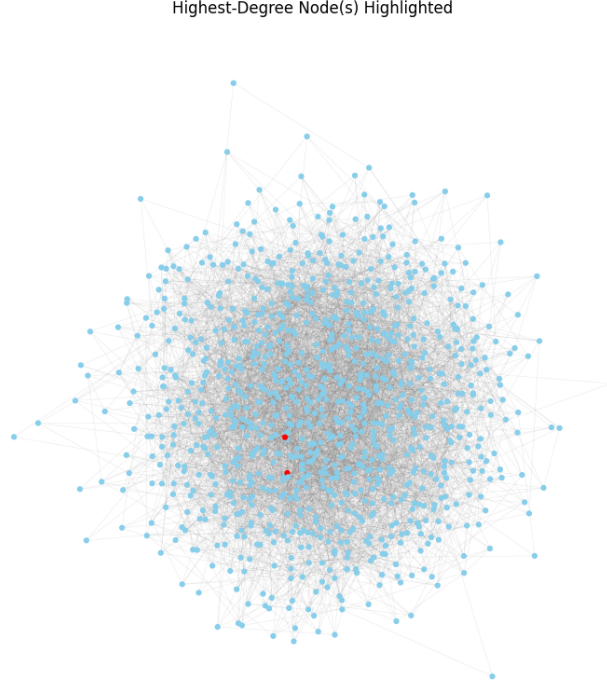


Figure 2: Generated graph with the highest-degree nodes highlighted.

The mean degree of the generated graph is

$$\bar{d} = \frac{1}{N} \sum_{i=1}^N d_i = 9.97$$

For an Erdős-Rényi graph $G(N, p)$, the theoretical mean degree is

$$\mathbb{E}[\bar{d}] = (N - 1)p = 999(0.01) = 9.99$$

Thus, the experimental and theoretical values are very close. The generated graph contains $|\mathcal{E}| = 4985$ edges, while the theoretical expected number of edges is

$$\mathbb{E}[|\mathcal{E}|] = p \binom{N}{2} = 0.01 \frac{1000(999)}{2} = 4995$$

The small difference between the experimental and theoretical values is expected due to the random nature of the Erdős-Rényi model.

The diameter of the graph is $\boxed{\text{diam}(\mathcal{G}) = 6}$. One pair of nodes achieving the diameter is (407, 762), with the corresponding shortest path

$$407 \rightarrow 345 \rightarrow 35 \rightarrow 402 \rightarrow 529 \rightarrow 683 \rightarrow 762$$

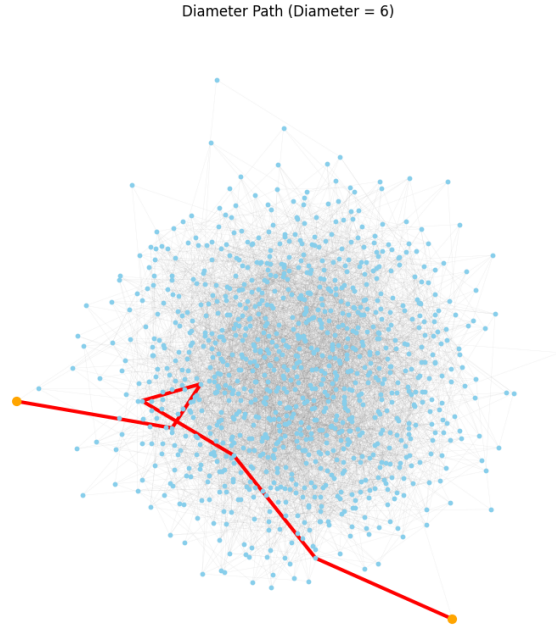


Figure 3: Diameter path of the generated graph highlighted.

(b) Shortest Path

Two nodes were selected randomly from the generated graph:

$$s = 89, \quad t = 773$$

Using Dijkstra's shortest-path algorithm, the shortest path between the two nodes was found to be

$$89 \rightarrow 380 \rightarrow 773$$

Hence, the shortest-path length is $d(89, 773) = 2$. The corresponding shortest path is highlighted in the graph below.

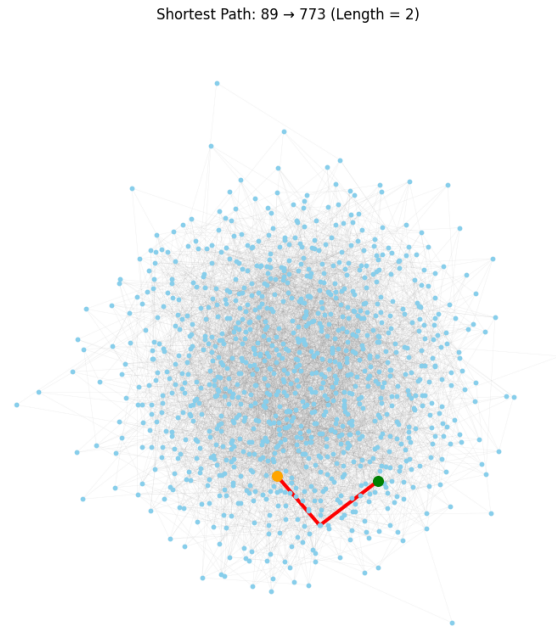


Figure 4: Shortest path between nodes 89 and 773.

Code for Problem 1

The following Python code was used to generate the graphs, compute the required graph matrices, properties and determine the shortest paths.

```
1 # Problem 1
2
3 import numpy as np
4 import networkx as nx
5 import matplotlib.pyplot as plt
6
7 # Reproducibility
8 SEED = 1
9
10 # Problem 1 parameters
11 N = 5
12 p = 0.5
13
14 # Generate Erd s-R nyi random undirected graph
15 G1 = nx.erdos_renyi_graph(N, p, seed=SEED)
16
17 print("Number of nodes:", G1.number_of_nodes())
18 print("Number of edges:", G1.number_of_edges())
19 print("Edges:", list(G1.edges()))
20
21 plt.figure(figsize=(6, 5))
22
23 pos = nx.spring_layout(G1, seed=SEED)
24
25 nx.draw(
26     G1,
27     pos,
28     with_labels=True,
29     node_size=800,
30     node_color="skyblue",
31     edge_color="gray",
32     font_size=12
33 )
34
35 plt.title(r"Erd s-R nyi Random Graph $G(5,0.5)$")
36 plt.axis("off")
37 plt.show()
38
39 # Problem 1(a): Graph Matrices
40
41 # Adjacency matrix
42 A = nx.to_numpy_array(G1, dtype=int)
43
44 # Degree matrix
45 degrees = np.array([G1.degree(i) for i in G1.nodes()])
46 D = np.diag(degrees)
47
48 # Laplacian matrix
49 L = D - A
50
51 # Incidence matrix
52 # Arbitrary orientation: first node = +1, second node = -1
53 edges = list(G1.edges())
54
55 B = np.zeros((N, len(edges)), dtype=int)
```

```

56
57 for j, (u, v) in enumerate(edges):
58     B[u, j] = 1
59     B[v, j] = -1
60
61 print("Adjacency Matrix A:")
62 print(A)
63
64 print("\nDegree Matrix D:")
65 print(D)
66
67 print("\nIncidence Matrix B:")
68 print(B)
69
70 print("\nLaplacian Matrix L:")
71 print(L)
72
73 # Problem 1(b): Laplacian Eigenvalues and Algebraic Connectivity
74
75 # Compute Laplacian eigenvalues
76 eigenvalues = np.linalg.eigvalsh(L)
77
78 # Sort eigenvalues
79 eigenvalues = np.sort(eigenvalues)
80
81 # Algebraic connectivity
82 lambda_2 = eigenvalues[1]
83
84 print("Laplacian eigenvalues:")
85 print(eigenvalues)
86
87 print("\nAlgebraic connectivity (lambda_2):")
88 print(lambda_2)
89
90 # Problem 2
91
92 # Problem 2 parameters
93 N2 = 1000
94 p2 = 0.01
95 SEED2 = 42
96
97 # Generate graphs until a connected graph is obtained
98 attempt = 0
99
100 while True:
101     G2 = nx.erdos_renyi_graph(N2, p2, seed=SEED2 + attempt)
102     if nx.is_connected(G2):
103         break
104     attempt += 1
105
106 print("Connected graph generated.")
107 print("Number of attempts:", attempt + 1)
108 print("Number of nodes:", G2.number_of_nodes())
109 print("Number of edges:", G2.number_of_edges())
110
111 plt.figure(figsize=(10, 10))
112
113 pos2 = nx.spring_layout(G2, seed=SEED2, iterations=30)
114
115 nx.draw_networkx_nodes(G2, pos2, node_size=8, node_color="steelblue")

```

```

116
117 nx.draw_networkx_edges(G2, pos2, width=0.3, alpha=0.3, edge_color="gray")
118
119 plt.title(r"Connected Erdős-Rényi Graph $G(1000,0.01)$")
120 plt.axis("off")
121 plt.show()
122
123 # Problem 2(a): Degree, Mean Degree, Mean Number of Edges, and Diameter
124
125 # Degree of every node
126 degree_dict = dict(G2.degree())
127 max_degree = max(degree_dict.values())
128 max_degree_nodes = [
129     node for node, degree in degree_dict.items()
130     if degree == max_degree
131 ]
132
133 mean_degree = np.mean(list(degree_dict.values()))
134 actual_edges = G2.number_of_edges()
135
136 # Theoretical values
137 theoretical_mean_degree = (N2 - 1) * p2
138 theoretical_mean_edges = p2 * N2 * (N2 - 1) / 2
139
140 print("Maximum degree:", max_degree)
141 print("Node(s) with maximum degree:", max_degree_nodes)
142
143 print("\nMean degree:")
144 print("Experimental:", mean_degree)
145 print("Theoretical:", theoretical_mean_degree)
146
147 print("\nNumber of edges:")
148 print("Experimental:", actual_edges)
149 print("Theoretical expected:", theoretical_mean_edges)
150
151 plt.figure(figsize=(10, 10))
152
153 # Draw all nodes
154 node_colors = [
155     "red" if node in max_degree_nodes else "skyblue"
156     for node in G2.nodes()
157 ]
158
159 nx.draw_networkx_nodes(G2, pos2, node_size=12, node_color=node_colors)
160
161 nx.draw_networkx_edges(G2, pos2, width=0.3, alpha=0.2, edge_color="gray")
162
163 plt.title("Highest-Degree Node(s) Highlighted")
164 plt.axis("off")
165 plt.show()
166
167 # Graph Diameter
168
169 diameter = nx.diameter(G2)
170 print("Graph diameter:", diameter)
171
172 diameter_pair = None
173
174 for source in G2.nodes():
175     lengths = nx.single_source_shortest_path_length(G2, source)

```

```

176     for target, distance in lengths.items():
177         if distance == diameter:
178             diameter_pair = (source, target)
179             break
180     if diameter_pair is not None:
181         break
182
183     print("Diameter pair:", diameter_pair)
184     diameter_path = nx.shortest_path(
185         G2,
186         source=diameter_pair[0],
187         target=diameter_pair[1]
188     )
189
190     # Convert path nodes into edges
191     diameter_edges = list(zip(diameter_path[:-1], diameter_path[1:]))
192
193     print("Diameter path:")
194     print(diameter_path)
195
196     plt.figure(figsize=(10, 10))
197
198     # Draw graph
199     nx.draw_networkx_nodes(G2, pos2, node_size=10, node_color="skyblue")
200
201     nx.draw_networkx_edges(G2, pos2, width=0.25, alpha=0.15, edge_color="gray")
202
203     # Highlight diameter edges
204     nx.draw_networkx_edges(G2, pos2, edgelist=diameter_edges, width=3, edge_color="red")
205
206     # Highlight diameter endpoints
207     nx.draw_networkx_nodes(G2, pos2, nodelist=list(diameter_pair), node_size=50,
208                           node_color="orange")
209
210     plt.title(f"Diameter Path (Diameter = {diameter})")
211     plt.axis("off")
212     plt.show()
213
214     # Problem 2(b): Shortest Path Between Two Random Nodes
215     rng = np.random.default_rng(SEED2)
216     source, target = rng.choice(list(G2.nodes()), size=2, replace=False)
217
218     print("Source node:", source)
219     print("Target node:", target)
220
221     # Compute shortest path using Dijkstra's algorithm
222     shortest_path = nx.shortest_path(
223         G2,
224         source=source,
225         target=target,
226         method="dijkstra"
227     )
228
229     shortest_distance = len(shortest_path) - 1
230
231     print("Shortest path:")
232     print(shortest_path)
233
234     print("\nShortest path length:", shortest_distance)

```

```

234
235 # Convert path into edges
236 shortest_path_edges = list(
237     zip(shortest_path[:-1], shortest_path[1:])
238 )
239
240 plt.figure(figsize=(10, 10))
241
242 # Draw all nodes
243 nx.draw_networkx_nodes(G2, pos2, node_size=10, node_color="skyblue")
244
245 # Draw all edges
246 nx.draw_networkx_edges(G2, pos2, width=0.25, alpha=0.15, edge_color="gray")
247
248 # Highlight shortest path
249 nx.draw_networkx_edges(G2, pos2, edgelist=shortest_path_edges, width=3,
250     edge_color="red")
251
252 # Highlight source and target
253 nx.draw_networkx_nodes(G2, pos2, nodelist=[source], node_size=70, node_color="
254     green")
255
256 nx.draw_networkx_nodes(G2, pos2, nodelist=[target], node_size=70, node_color="
257     orange")
258
259 plt.title(
260     f"Shortest Path: {source}          {target} "
261     f"(Length = {shortest_distance})"
262 )
263
264 plt.axis("off")
265 plt.show()

```

Problem 2

We denote the i^{th} component of a vector x at time $k \in \mathbb{N}$ as $(x_k)_i$. Consider a linear system given by $x_{t+1} = Ax_t$, $t = 1, \dots, T$, with $x_t \in \mathbb{R}^n$ and $A \in \mathbb{R}^{n \times n}$. Find the matrix $S \in \mathbb{R}^{n \times n}$ that depends upon the matrix A to measure the n numbers $\{(x_1)_1, (x_2)_2, \dots, (x_n)_n\}$, i.e., the i^{th} component of the state at time i , for $i = 1, \dots, n$, such that

$$\begin{bmatrix} (x_1)_1 \\ (x_2)_2 \\ \vdots \\ (x_n)_n \end{bmatrix} = Sx_1.$$

Solution:

From the system equation, $x_{t+1} = Ax_t$, we obtain recursively $x_2 = Ax_1, x_3 = Ax_2 = A^2x_1$ and in general: $x_i = A^{i-1}x_1$.

Let e_i denote the i^{th} standard basis vector. Then the i^{th} component of x_i can be written as $(x_i)_i = e_i^T x_i$. Using $x_i = A^{i-1}x_1$,

$$(x_i)_i = e_i^T A^{i-1}x_1$$

Therefore, $\begin{bmatrix} (x_1)_1 \\ (x_2)_2 \\ \vdots \\ (x_n)_n \end{bmatrix} = \begin{bmatrix} e_1^T \\ e_2^T A \\ \vdots \\ e_n^T A^{n-1} \end{bmatrix} x_1$. Comparing with $\begin{bmatrix} (x_1)_1 \\ (x_2)_2 \\ \vdots \\ (x_n)_n \end{bmatrix} = Sx_1$, we obtain $S = \begin{bmatrix} e_1^T \\ e_2^T A \\ \vdots \\ e_n^T A^{n-1} \end{bmatrix}$

Hence, the i^{th} row of S is $\boxed{S_{i,:} = e_i^T A^{i-1}}$.

Problem 3

Let

$$A = \begin{bmatrix} B & I_n \\ 0_n & B \end{bmatrix}, \quad B \in \mathbb{R}^{n \times n}.$$

Compute the following.

1. Find e^{At} .
2. Let $n = 2$ with $\alpha \in \mathbb{R}$, $\beta > 0$ and assume

$$B = \begin{bmatrix} 1 & \beta \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \alpha & 1 \\ 0 & \alpha \end{bmatrix} \begin{bmatrix} 1 & -\beta \\ 0 & 1 \end{bmatrix}.$$

Compute e^{At} .

Solution:

1. Find e^{At}

Let $A = \begin{bmatrix} B & I_n \\ 0_n & B \end{bmatrix}$

For $k \geq 1$, we have

$$\begin{aligned} A^k &= \begin{bmatrix} B & I_n \\ 0_n & B \end{bmatrix}^k \\ &= \begin{bmatrix} B^k & kB^{k-1} \\ 0_n & B^k \end{bmatrix} \end{aligned}$$

Using the definition of the matrix exponential,

$$\begin{aligned} e^{At} &= I_{2n} + \sum_{k=1}^{\infty} \frac{A^k t^k}{k!} \\ &= \begin{bmatrix} I_n & 0_n \\ 0_n & I_n \end{bmatrix} + \sum_{k=1}^{\infty} \frac{t^k}{k!} \begin{bmatrix} B^k & kB^{k-1} \\ 0_n & B^k \end{bmatrix} \\ &= \begin{bmatrix} I_n + \sum_{k=1}^{\infty} \frac{B^k t^k}{k!} & \sum_{k=1}^{\infty} \frac{kB^{k-1} t^k}{k!} \\ 0_n & I_n + \sum_{k=1}^{\infty} \frac{B^k t^k}{k!} \end{bmatrix} \\ &= \begin{bmatrix} e^{Bt} & te^{Bt} \\ 0_n & e^{Bt} \end{bmatrix} \end{aligned}$$

Hence,

$$\boxed{e^{At} = \begin{bmatrix} e^{Bt} & te^{Bt} \\ 0_n & e^{Bt} \end{bmatrix}}.$$

2. Let $n = 2$

Given $B = \begin{bmatrix} 1 & \beta \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \alpha & 1 \\ 0 & \alpha \end{bmatrix} \begin{bmatrix} 1 & -\beta \\ 0 & 1 \end{bmatrix},$

define $P = \begin{bmatrix} 1 & \beta \\ 0 & 1 \end{bmatrix}$, $J = \begin{bmatrix} \alpha & 1 \\ 0 & \alpha \end{bmatrix}$

Then

$$B = PJP^{-1},$$

where

$$P^{-1} = \begin{bmatrix} 1 & -\beta \\ 0 & 1 \end{bmatrix}$$

Therefore, $e^{Bt} = Pe^{Jt}P^{-1}$

Now, $J = \alpha I_2 + \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$, and $\begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}^2 = 0$

Thus,

$$\begin{aligned} e^{Jt} &= e^{\alpha t} \exp \left(t \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \right) \\ &= e^{\alpha t} \begin{bmatrix} 1 & t \\ 0 & 1 \end{bmatrix} \end{aligned}$$

Hence,

$$\begin{aligned} e^{Bt} &= e^{\alpha t} \begin{bmatrix} 1 & \beta \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & -\beta \\ 0 & 1 \end{bmatrix} \\ &= e^{\alpha t} \begin{bmatrix} 1 & t + \beta \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & -\beta \\ 0 & 1 \end{bmatrix} \\ &= e^{\alpha t} \begin{bmatrix} 1 & t \\ 0 & 1 \end{bmatrix} \end{aligned}$$

Therefore,

$$te^{Bt} = e^{\alpha t} \begin{bmatrix} t & t^2 \\ 0 & t \end{bmatrix}$$

Using the result from part 1,

$$\begin{aligned} e^{At} &= \begin{bmatrix} e^{Bt} & te^{Bt} \\ 0_2 & e^{Bt} \end{bmatrix} \\ &= e^{\alpha t} \begin{bmatrix} 1 & t & t & t^2 \\ 0 & 1 & 0 & t \\ 0 & 0 & 1 & t \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{aligned}$$

Hence,

$$e^{At} = e^{\alpha t} \begin{bmatrix} 1 & t & t & t^2 \\ 0 & 1 & 0 & t \\ 0 & 0 & 1 & t \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Problem 4

Given an interval $[a, b]$ with $a < b$ and a polynomial

$$p(x) = \sum_{k=0}^{n-1} c_{k+1} x^k$$

with coefficients $c \in \mathbb{R}^n$, prove that both integral and derivative of the polynomial $p(x)$ is a linear

function of its coefficients. That is, prove that $\exists \alpha \in \mathbb{R}^n$, $\exists \beta \in \mathbb{R}^n$ such that

$$\alpha^T c = \int_a^b p(x) dx$$

and

$$\beta^T c = p'(x) = \frac{dp(x)}{dx}.$$

Solution:

Given $p(x) = \sum_{k=0}^{n-1} c_{k+1} x^k$, we can write

$$p(x) = \begin{bmatrix} 1 & x & x^2 & \cdots & x^{n-1} \end{bmatrix} c$$

1. Integral of $p(x)$

Integrating term by term,

$$\begin{aligned} \int_a^b p(x) dx &= \int_a^b \sum_{k=0}^{n-1} c_{k+1} x^k dx \\ &= \sum_{k=0}^{n-1} c_{k+1} \int_a^b x^k dx \\ &= \sum_{k=0}^{n-1} c_{k+1} \frac{b^{k+1} - a^{k+1}}{k+1} \end{aligned}$$

Therefore,

$$\int_a^b p(x) dx = \begin{bmatrix} b-a & \frac{b^2-a^2}{2} & \cdots & \frac{b^n-a^n}{n} \end{bmatrix} c$$

Hence,

$$\alpha = \begin{bmatrix} b-a \\ \frac{b^2-a^2}{2} \\ \vdots \\ \frac{b^n-a^n}{n} \end{bmatrix}$$

and

$$\alpha^T c = \int_a^b p(x) dx$$

2. Derivative of $p(x)$

Differentiating term by term,

$$\begin{aligned} p'(x) &= \frac{d}{dx} \sum_{k=0}^{n-1} c_{k+1} x^k \\ &= \sum_{k=1}^{n-1} k c_{k+1} x^{k-1} \end{aligned}$$

Therefore, $p'(x) = \begin{bmatrix} 0 & 1 & 2x & \cdots & (n-1)x^{n-2} \end{bmatrix} c$

Hence, defining

$$\beta(x) = \begin{bmatrix} 0 \\ 1 \\ 2x \\ \vdots \\ (n-1)x^{n-2} \end{bmatrix}$$

we obtain

$$\beta(x)^T c = p'(x)$$

Thus, both the integral and derivative of $p(x)$ are linear functions of its coefficient vector c .

Problem 5

Let $\{u_i \mid i \in \mathbb{N}_{\geq 1}\}$ be a sequence of vectors. Given $x_1 = 0$, let

$$x_t = \frac{1}{t-1} \sum_{k=1}^{t-1} u_k, \quad \forall t \in \mathbb{N}_{\geq 2}$$

This represents a recursive mean computation of a collection of vectors by accessing them one by one. Find A, B matrices such that

$$x_{t+1} = Ax_t + Bu_t, \quad t = 1, 2, \dots$$

Solution:

Given $x_t = \frac{1}{t-1} \sum_{k=1}^{t-1} u_k$, $t \geq 2$

For x_{t+1} ,

$$\begin{aligned} x_{t+1} &= \frac{1}{t} \sum_{k=1}^t u_k \\ &= \frac{1}{t} \left(\sum_{k=1}^{t-1} u_k + u_t \right) \end{aligned}$$

From the definition of x_t , $\sum_{k=1}^{t-1} u_k = (t-1)x_t$

Therefore,

$$\begin{aligned} x_{t+1} &= \frac{1}{t} ((t-1)x_t + u_t) \\ &= \frac{t-1}{t} x_t + \frac{1}{t} u_t \end{aligned}$$

Comparing with $x_{t+1} = Ax_t + Bu_t$, we obtain

$$A_t = \frac{t-1}{t} I, \quad B_t = \frac{1}{t} I$$

Hence, the recursive mean can be written as

$$x_{t+1} = \frac{t-1}{t} I x_t + \frac{1}{t} I u_t$$

Thus, A and B are time-varying matrices

AI Usage Declaration

AI tools were used after the solutions and results had been obtained through mathematical work and python code. They were used to improve the formatting, readability and clarity of the explanations